

DevNexus | AgileBridge

API CONTRACTS

Prepared By
DevNexus



devnexus28@gmail.com

 agilebridge
CONSIDERED
SOFTWARE SOLUTIONS

Auth API

Base path: /api/auth

POST /register

Registers a new student account and triggers OTP delivery to their university email.

Request body

```
{
  "firstName": "string",
  "lastName": "string",
  "email": "string",
  "phoneNumber": "string | null",
  "password": "string",
  "yearOfStudy": "number",
  "universityId": "number",
  "degreeProgram": "string"
}
```

Responses

Status	Body	When
200	"OTP sent to your email."	Registered, OTP dispatched
409	"email_taken"	Email already registered and verified
422	"invalid_email"	Email fails format validation
422	"invalid_year_of_study"	yearOfStudy not between 1 and 10
422	"invalid_domain"	Email domain not a recognised university, or fails IDNA encoding
422	"weak_password"	Password fails strength check
429	"otp_already_sent"	Existing pending registration for this email
500	"server_error"	Unhandled exception Rate limited (policy: register).

POST /verify-otp

Verifies the OTP sent during registration.

Request body { "email": "string", "otp": "string" }

Responses

Status	Body	When
200	"Verified successfully."	OTP correct
401	"invalid_otp"	No user found for email Rate limited (policy: verify-otp).

POST /resend-otp

Resends a pending registration's OTP.

Request body { "email": "string" }

Responses

Status	Body	When
200	"If this email is registered and pending verification, a new OTP has been sent."	Always Rate limited (policy: resend-otp).

POST /login

Authenticates a user and sets an HttpOnly session cookie.

Request body { "email": "string", "password": "string" }

Responses

Status	Body	When
200	"Login successful"	Credentials valid
401	"invalid_credentials"	Missing email/password, unknown email, deleted account, or password mismatch
500	"server_error"	Unhandled exception Rate limited (policy: login).

POST /logout

Clears the session cookie. No request body.

Responses

Status	Body	When
200	"Logged out successfully"	Always

GET /me

Returns the authenticated user's profile. Requires authentication.

Responses

Status	Body	When
200	MeResponse	Authenticated
401	"unauthenticated"	No user ID claim on the request
401	"unauthenticatedfailing"	User ID claim present but no matching user found
500	"server_error"	Unhandled exception

GET /hub-token

Issues a short-lived JWT scoped for the SignalR hub connection.

Responses

Status	Body	When
200	{ "token": "string" }	Authenticated
401	-	No user ID claim on the request

Shared types MeResponse (student accounts, role: "student") { "user": { "userId": "guid", "firstName": "string", "lastName": "string", "email": "string", "phoneNumber": "string", "userRole": "string" }, "std": { "verificationStatus": "string", "degreeProgram": "string", "yearOfStudy": "number", "university": "string", "sellerTrustScore": "number", "buyerReliabilityScore": "number" }}

MeResponse (non-student accounts) { "userId": "guid", "firstName": "string", "lastName": "string", "email": "string", "phoneNumber": null, "userRole": null }

Password Rules Minimum 8 characters; requires at least one uppercase, one lowercase, one digit, and one non-alphanumeric character.

Course API

Base path: /api/courses

GET /

Searches courses by name/code, optionally filtered by university. Requires authentication.

Query params

- search (string, optional)
- universityId (number, optional)
- limit (number, optional, default: 20)

Responses

Status	Body	When
200	CourseDto[]	Always

GET /{id}

Returns a single course by ID. Requires authentication.

Path params — id: number

Responses

Status	Body	When
200	CourseDto	Found
404	{ "error": "course_not_found" }	Not found

Shared types CourseDto { "courseId": "number", "courseCode": "string", "courseName": "string", "faculty": "string" }

Dev API

Base path: /api/dev

GET /otp

Retrieves the most recently issued OTP for a given email. Development environment only.

Query params

- email (string, required)

Responses

Status	Body	When
200	{ "otp": "string" }	Development environment, and an OTP has been issued
404	-	Not running in Development environment
404	{ "error": "no_otp_found" }	No OTP found for this email

DeviceTokens API

Base path: /api/device-tokens

POST /

Registers a push-notification device token for the authenticated user. Requires authentication.

Request body { "token": "string", "platform": "string" }

Responses

Status	Body	When
200	{ "message": "Token registered" }	Registered successfully
400	{ "error": "token_required" }	token is null, empty, or whitespace
400	{ "error": "invalid_platform" }	platform is not one of "web", "ios", "android"
401	{ "error": "invalid_user_claim" }	Authenticated but user ID claim couldn't be parsed as a GUID

Listing API

Base path: /api/listings

POST /

Creates a new listing. Requires authentication.

Request body { "title": "string", "description": "string", "price": "number", "condition": "string", "categoryName": "string", "metadata": "object | null", "bookDetails": { "isbn": "string | null", "author": "string | null", "edition": "string | null" }, "listingStatus": "string", "courseId": "number | null", "isBundle": "boolean", "images": [{ "imageUrl": "string", "isPrimary": "boolean" }] }

Responses

Status	Body	When
200	ListingSummary	Created successfully
400	{ "error": "Field(s) missing." }	Missing title or invalid condition/price when not a draft
400	{ "error": "invalid_category" }	Provided category name is invalid
400	{ "error": "book_fileds_not_allowed" }	Book details provided for a non-book category
400	{ "error": "invalid_metadata" }	Category metadata validation failed
401	{ "error": "unauthenticated" }	Missing or invalid user authorization context

PUT /{id}

Updates an existing listing by ID. Requires authentication. Path params — id: guid

Request body { "title": "string", "description": "string", "price": "number", "condition": "string", "categoryName": "string | null", "courseId": "number | null", "metadata": "object | null", "bookDetails": { "isbn": "string | null", "author": "string | null", "edition": "string | null" }, "removedImageIds": "number[]" | null, "isBundle": "boolean | null" }

Responses

Status	Body	When
200	"Listings updated successfully"	Updated successfully
401	{ "error": "unauthenticated" }	Missing or invalid user authorization context
404	-	Listing not found or user is not owner

DELETE /{id}

Deletes a listing by ID. Requires authentication. Path params — id: guid

Responses

Status	Body	When
204	-	Deleted successfully
401	{ "error": "unauthenticated" }	Missing or invalid user authorization context
403	{ "error": "forbidden" }	User does not have permission to delete this listing
404	{ "error": "listing_not_found" }	Listing not found

GET /

Returns a paginated list of listings with optional filters. Requires authentication.

Query params

- categoryId (number | null)
- listingType (string | null)
- listingStatus (string | null)
- courseId (number | null)
- sellerId (guid | null)
- excludeSellerId (guid | null)
- search (string | null)
- skip (number, default: 0)
- take (number, default: 50)

Responses

Status	Body	When
200	PagedResult	Retrieved successfully

GET /{id}

Returns a single listing by ID. Requires authentication. Path params — id: guid

Responses

- Status * Body * When
- 200 * ListingSummary * Found
- 404 * { "error": "listing_not_found" } * Not found

POST /{listingId}/images

Uploads one or more images for a specific listing owned by the caller. Requires authentication.

Path params — listingId: guid Content-Type: multipart/form-data

Form fields

- files (File[]): JPEG, PNG, or WebP only. Max 10MB per file.

Responses

Status	Body	When
200	{ "imageIds": "number[]" }	All files uploaded
400	"no_files"	No files provided
400	"file_too_large"	A file exceeds 10MB or is 0 bytes
400	"invalid_file_type"	File isn't JPEG, PNG, or WebP
401	{ "error": "unauthenticated" }	Missing or invalid authorization
403	{ "error": "forbidden" }	User is not the owner of this listing

GET /{listingId}/images/{imageId}

Retrieves a binary image file associated with a listing. Requires authentication. Path params

- listingId: guid
- imageId: number

Responses

Status	Body	When
200	image binary data	Image found
304	-	Not modified
404	-	Image not found

PATCH /{id}/status

Updates the status of a listing. Requires authentication. Path params — id: guid

Request body { "status": "string" }

Responses

Status	Body	When
200	{ "status": "string" }	Status updated successfully
401	{ "error": "unauthenticated" }	Missing or invalid authorization
403	{ "error": "forbidden" }	User is not authorized to update this listing
404	{ "error": "listings_not_found" }	Listing not found
409	{ "error": "status_locked" }	Status transition is invalid or locked
409	{ "error": "images_required" }	Cannot publish listing without images
409	{ "error": "description_required" }	Cannot publish listing without description

Shared types ListingSummary { "listingId": "guid", "sellerId": "guid", "title": "string", "description": "string", "price": "number", "condition": "string", "categoryName": "string", "courseId": "number | null", "bookDetails": { "isbn": "string | null", "author": "string | null", "edition": "string | null" }, "metadata": "object | null", "listingStatus": "string", "isBundle": "boolean", "viewCount": "number", "createdAt": "datetime", "updatedAt": "datetime", "images": [{ "imageId": "number", "url": "string", "isPrimary": "boolean" }] }

PagedResult { "items": "ListingSummary[]", "total": "number" }

ListingCategories API Base path: /api/listing-categories

GET /

Returns a list of all active listing categories. Public / Unauthenticated.

Responses

Status	Body	When
200	CategorySummary[]	Retrieved successfully

Shared types CategorySummary { "id": "number", "name": "string" }

Reservations Base path: /api/reservations

POST /

Creates a new listing reservation. Requires verified user authorization.

Request body { "listingId": "guid" }

Responses

Status	Body	When
201	ReservationDto	Reservation created successfully
403	{ "error": "not_verified" }	User is not verified
403	{ "error": "self_reserve" }	User attempted to reserve own listing
403	{ "error": "forbidden" }	User does not have access
404	{ "error": "listing_not_found" }	Listing specified by ID does not exist
409	{ "error": "already_reserved" }	Listing is already reserved
500	{ "error": "server_error" }	Internal server error

POST /{id}/acknowledge

Acknowledges a reservation. Requires authorization. Path params - id: guid

Responses

Status	Body	When
200	ReservationDto	Acknowledged successfully
403	{ "error": "forbidden" }	User does not have authorization
404	{ "error": "not_found" }	Reservation not found
409	{ "error": "already_acknowledged" }	Reservation was already acknowledged
409	{ "error": "already_terminal" }	Reservation is completed or canceled
500	{ "error": "server_error" }	Internal server error

POST /{id}/cancel

Cancels an active reservation. Requires authorization. Path params - id: guid

Responses

Status	Body	When
200	ReservationDto	Canceled successfully
403	{ "error": "forbidden" }	User not permitted to cancel
404	{ "error": "not_found" }	Reservation not found
409	{ "error": "already_terminal" }	Reservation is already terminated
500	{ "error": "server_error" }	Internal server error

GET /

Returns a list of reservations for the authenticated user based on role. Requires authorization.

Query params

- role (string, required): Must be "buyer" or "seller"

Responses

Status	Body	When
200	{ "items": "ReservationListItemDto[]", "hasMore": false, "nextCursor": null }	Retrieved
400	{ "error": "invalid_role" }	Role parameter invalid

GET /{id}

Retrieves a reservation by ID. Requires authorization. Path params - id: guid

Responses

Status	Body	When
200	ReservationDto	Found
404	{ "error": "not_found" }	Reservation not found

GET /{id}/messages

Gets paginated chat message history associated with a reservation. Requires authorization.

Path params - id: guid

Query params

- before (number | null)
- limit (number, default: 20, max: 100)

Responses

Status	Body	When
200	ChatMessageDto	Messages retrieved successfully
403	{ "error": "forbidden" }	User is not part of this reservation

Meetups API

Base path: /api/reservations/{reservationId}/meetup

POST /propose

Proposes a meetup location and time for a reservation. Requires authorization. Path params - reservationId: guid

Request body

```
{  
  "locationName": "string",  
  "lat": "number",  
  "lng": "number",  
  "proposedTime": "datetime"  
}
```

Responses

Status	Body	When
200	ChatMessageDto	Proposal created successfully
400	{ "error": "time_in_past" }	Proposed time is in the past
403	{ "error": "forbidden" }	User is not authorized
404	{ "error": "not_found" }	Reservation not found
409	{ "error": "not_active" }	Reservation is not in active state
500	{ "error": "server_error" }	Internal server error

POST /accept

Accepts a proposed meetup location/time. Requires authorization.

Path params - reservationId: guid

Request body

```
{  
  "proposalMessageId": "number"  
}
```

Responses

Status	Body	When
200	MeetupResponseResult	Meetup accepted and scheduled
400	{ "error": "not_a_proposal" }	Target message is not a meetup proposal
403	{ "error": "cannot_accept_own_proposal" }	User cannot accept own proposal
404	{ "error": "proposal_not_found" }	Proposal message not found
409	{ "error": "already_responded" }	Proposal was already accepted or declined
500	{ "error": "server_error" }	Internal server error

POST /decline

Declines a proposed meetup location/time. Requires authorization.

Path params - reservationId: guid

Request body

```
{  
  "proposalMessageId": "number"  
}
```

Responses

Status	Body	When
200	MeetupResponseResult	Proposal declined
400	{ "error": "not_a_proposal" }	Message is not a proposal
404	{ "error": "proposal_not_found" }	Proposal not found
409	{ "error": "already_responded" }	Proposal was already responded to
500	{ "error": "server_error" }	Internal server error

POST /check-in

Performs user check-in at the scheduled meetup. Requires authorization.

Path params - reservationId: guid

Request body

```
{  
  "lat": "number | null",  
  "lng": "number | null"
```

}

Responses

Status	Body	When
200	CheckInResult	Check-in registered
404	{ "error": "meetup_not_found" }	Meetup record missing
409	{ "error": "meetup_not_scheduled" }	Meetup is not scheduled
409	{ "error": "already_checked_in" }	User already checked in
409	{ "error": "check_in_window_closed" }	Outside allowed check-in window
500	{ "error": "server_error" }	Internal server error

GET /

Returns current status and check-in details of a scheduled meetup. Requires authorization.

Path params — reservationId: guid

Responses

Status	Body	When
200	MeetupStatusDto	Retrieved successfully
404	{ "error": "meetup_not_found" }	Meetup status not found

Shared Types

ReservationDto

{

"reservationId": "guid",

"listingId": "guid",

"buyerId": "guid",

"sellerId": "guid",

"reservationStatus": "string",

"timerStage": "string",

"expiresAt": "datetime",

"createdAt": "datetime",

"completedAt": "datetime | null",

"counterParty": {

"userId": "guid",

"name": "string",

"initials": "string"

}

}

ReservationListItemDto

```
{
  "reservationId": "guid",
  "reservationStatus": "string",
  "timerStage": "string",
  "expiresAt": "datetime",
  "createdAt": "datetime",
  "counterParty": {
    "userId": "guid",
    "name": "string",
    "initials": "string"
  },
  "listing": {
    "listingId": "guid",
    "title": "string",
    "price": "number",
    "imagePath": "string | null"
  },
  "unreadCount": "number",
  "lastMessagePreview": "string | null",
  "lastMessageAt": "datetime | null"
}
```

MeetupResponseResult

```
{
  "responseMessage": "ChatMessageDto",
  "meetupId": "number | null",
  "reservation": "ReservationDto"
}
```

CheckInResult

```
{
  "meetupId": "number",
```

```
"checkedIn": "boolean",
"locationVerified": "boolean",
"checkInAt": "datetime",
"paymentUnlocked": "boolean",
"message": "string"
}

MeetupStatusDto
{
"meetupId": "number",
"agreedLocationName": "string",
"agreedLatitude": "number",
"agreedLongitude": "number",
"agreedTime": "datetime",
"createdAt": "datetime",
"checkinWindowClosesAt": "datetime",
"checkinWindowOpensAt": "datetime",
"checkInWindowOpen": "boolean",
"buyerCheckedIn": "boolean",
"buyerCheckedInAt": "datetime | null",
"sellerCheckedIn": "boolean",
"sellerCheckedInAt": "datetime | null",
"paymentUnlocked": "boolean",
"status": "string"
}
```

Reviews API

Base path: /api/reviews

POST /

Creates a new review for a completed transaction. Requires authorization.

Request body

```
{
"transactionId": "guid",
```

```
"rating": "number",  
"comment": "string | null"  
}
```

Responses

Status	Body	When
200	ReviewDto	Review created successfully
400	{ "error": "invalid_rating" }	Rating value is invalid
403	{ "error": "not_a_party" }	User was not a buyer or seller in transaction
403	{ "error": "self_review" }	User attempted to review themselves
404	{ "error": "transaction_not_found" }	Transaction not found
409	{ "error": "transaction_not_complete" }	Transaction status is not completed
409	{ "error": "already_reviewed" }	Review already exists for this transaction
500	{ "error": "server_error" }	Internal server error

GET /users/{userId}

Retrieves all reviews associated with a specific user. Requires authorization.

Path params - userId: guid

Responses

Status	Body	When
200	ReviewDto[]	Reviews retrieved successfully

Transactions API

Base path: /api/reservations

POST /{reservationId}/Transaction-request

Initiates a payment transaction request for a reservation. Requires authorization.

Path params - reservationId: guid

Responses

Status	Body	When
200	{ "sandbox_url": "string", "fields": "object" }	Request processed successfully
401	-	Missing/invalid authentication claim
403	-	User is not the buyer for this reservation
404	{ "code": "reservation_not_found" }	Reservation ID not found

400	{ "code": "invalid_status" }	Reservation is not in an active state
-----	------------------------------	---------------------------------------

POST /itn

Instant Transaction Notification webhook endpoint.

Headers / Auth: Public / Webhook

Content-Type: application/x-www-form-urlencoded

Payload Parameters

- signature (string, required)
- m_payment_id (guid / reservationId, required)
- pf_payment_id (string)
- payment_status (string)

Responses

Status	Body	When
200	-	Notification processed successfully
400	-	Missing signature parameter
400	"invalid_signature"	Signature verification failed
400	"invalid_payment_id"	Missing or unparseable m_payment_id
404	-	Reservation not found

POST /{reservationId}/verify-pin

Verifies the PIN entered by the buyer to complete transaction payout. Requires authorization.

Path params - reservationId: guid

Request body

```
{
  "pin": "string"
}
```

Responses

Status	Body	When
200	-	PIN verified successfully
401	-	Missing/invalid authentication claim
403	-	User is not the buyer
404	{ "code": "transaction_not_found" }	Transaction not found
400	{ "code": "too_many_attempts" }	Max PIN attempts exceeded
400	{ "code": "invalid_pin" }	PIN is incorrect

GET /{reservationId}/transaction-status

Gets current transaction and PIN status for a reservation. Requires authorization.

Path params - reservationId: guid

Responses

Status	Body	When
200	{ "transactionStatus": "none", "pinStatus": null, "pin": null }	No transaction found
200	{ "transactionId": "guid", "transactionStatus": "string", "pinStatus": "string", "pin": "string null" }	Transaction status details found
401	-	Missing/invalid authentication claim
403	-	User is neither buyer nor seller

GET /{reservationId}/pending-pin

Retrieves the active PIN for the seller to provide to the buyer. Requires authorization.

Path params - reservationId: guid

Responses

Status	Body	When
200	{ "pin": "string" }	PIN retrieved successfully
401	-	Missing/invalid authentication claim
403	-	User is not the seller
404	{ "code": "reservation_not_found" }	Reservation not found
404	{ "code": "transaction_not_found" }	Transaction record missing

University API

Base path: /api/universities

GET /

Retrieves a list of all active universities. Public / Unauthenticated.

Responses

Status	Body	When
200	{ "count": "number", "data": "University[]" }	Active universities retrieved successfully

Users API

Base path: /api/users

GET /me

Retrieves profile information for the currently authenticated user. Requires authorization.

Responses

Status	Body	When
200	UserDto	Profile data retrieved successfully
401	{ "error": "unauthenticated" }	Missing or unparseable user ID claim
401	{ "error": "user_not_found" }	User account does not exist
500	{ "error": "server_error" }	Internal server error

PUT /profile

Updates the profile details of the authenticated user. Requires authorization.

Request body

```
{  
  "degreeProgram": "string | null",  
  "yearOfStudy": "number | null"  
}
```

Responses

Status	Body	When
200	UserDto	Profile updated successfully
401	{ "error": "unauthenticated" }	Missing or unparseable user ID claim
404	{ "error": "user_not_found" }	User account does not exist
422	{ "error": "invalid_year_of_study" }	Provided year of study value is invalid
422	{ "error": "degree_program_required" }	Degree program field is required
500	{ "error": "server_error" }	Internal server error

DELETE /account

Deletes the account of the authenticated user and clears the authentication cookie. Requires authorization.

Responses

Status	Body	When
200	{ "message": "Account deleted successfully" }	Account deleted successfully
401	{ "error": "unauthenticated" }	Missing or unparseable user ID claim
404	{ "error": "user_not_found" }	User account does not exist
500	{ "error": "server_error" }	Internal server error

Wishlist API

Base path: /api/wishlist

POST /

Adds a listing to the authenticated user's wishlist. Requires authorization.

Request body { "listingId": "guid" }

Responses

Status	Body	When
200	WishlistItemDto	Item added to wishlist successfully
404	{ "error": "listing_not_found" }	Listing ID does not exist
404	{ "error": "not_found" }	Target item not found
409	{ "error": "already_wishlisted" }	Listing is already in the wishlist
409	{ "error": "listing_unavailable" }	Listing is unavailable
500	{ "error": "server_error" }	Internal server error

GET /

Retrieves all wishlist items for the authenticated user. Requires authorization.

Responses

Status	Body	When
200	{ "items": "WishlistItemDto[]", "total": "number" }	Wishlist items retrieved successfully

DELETE /{listingId}

Removes a listing from the authenticated user's wishlist. Requires authorization. Path params - listingId: guid

Responses

Status	Body	When
204	-	Item removed successfully
404	{ "error": "not_found" }	Wishlist item not found